

データベース演習

第7回：SQLによる集計

鈴木源吾

前回の復習

1. 解説：SQLによるデータ分析
 - 現実的な分析の方法とは
 - データ分析に用いるSQL
2. 例題：サンプルデータのSQLによる検索
 - サンプルデータベースの作成
 - SQLによる検索
3. 演習：サンプルデータのSQLによる検索

今回のトピック

1. 集約関数による単純集計

- 集約関数による単純集計
- 例題・演習

2. GROUP BY句の利用

- GROUP BY句で期間ごとに集計
- GROUP BY句でクロス集計？
- 例題・演習

参考資料：その他の資料「10年戦えるデータ分析入門」第4章

1. 集約関数による単純集計

SQLによる集計

データ分析の基礎は，集計である

- 事前に様々な集計をしてデータの性質をつかむ
- 合計や平均のような単純な集計のみで問題が解決することも多い
- SQLには集計に便利な関数や機能がいろいろ入っている

集約関数による単純集計

SQLでテーブルの値を集計するには、**集約関数**を使う

- 集約関数は、テーブルの複数の行にまたがって、合計や平均などの計算を「縦方向に」行うことができる
- 平均を求める例を以下にあげる（データの値はサンプルDBと異なる）
 - テーブル access_log_wideは、access_logにcustomers（顧客）のラムなどを結合したもの。ログを記録した顧客をユーザと呼ぶ

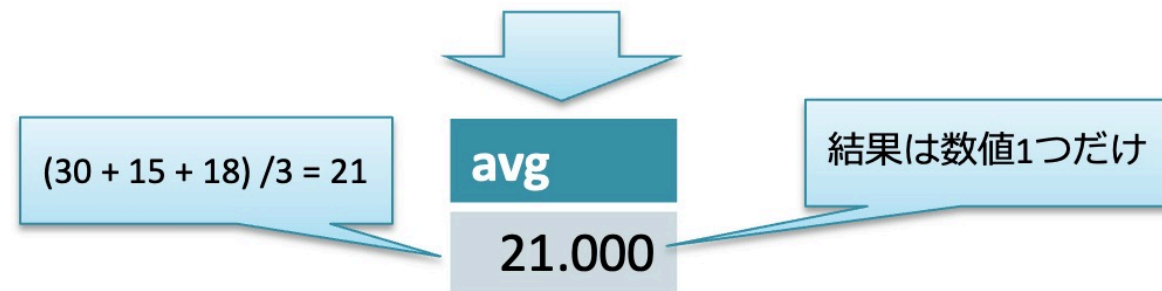
access_log_wide

「縦方向に」集計する

request_time	customer_age
2014-12-29 03:34:01	30
2014-12-29 03:34:05	15
2014-12-29 03:34:34	18

ユーザの年齢の平均を求めるSQL

```
SELECT avg(customer_age) FROM access_log_wide;
```



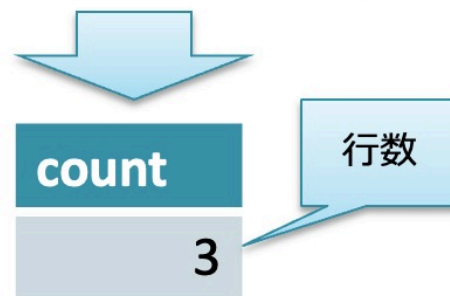
さまざまな集約関数

- 下図のような様々な集約関数がある
 - 前回に出てきた行数を数えるcountも集約関数の一種
- 関数のカッコの中は、関数の**引数**と呼ばれ、カラム名を指定する
 - 引数のカラムを変えることによって、様々なカラムの集計を行える

access_log_wide

request_time	customer_age
2014-12-29 03:34:01	30
2014-12-29 03:34:05	15
2014-12-29 03:34:34	18

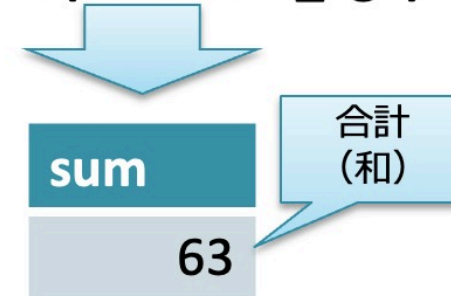
count(customer_age)



min(customer_age)



sum(customer_age)



max(customer_age)



集約関数の利用例

```
SELECT avg(search_hit) FROM access_log_wide;
```

- 全行のsearch_hitカラムの値の平均値を計算する
- この例のsearch_hitカラムは，検索のときにヒットした件数を表す
 - request_pathが/searchのとき（つまり検索）のときだけ値がある
 - avg(search_hit)は，検索で平均何件ヒットしたかを示す
- 1ヶ月ごとにsearch_hitの平均値を出したときに値が上向いていれば，よりたくさんの記事が検索でヒットするようになってきていることがわかる

検索結果

avg

154.000

集約関数を使うときの注意

以下のような書き方はできない

```
SELECT avg(search_hit), search_hit FROM access_log_wide;
```

以下のエラーが出る

- ERROR: column "access_log_wide.search_hit" must appear in the GROUP BY clause or be used in an aggregate function
- 訳：カラム"access_log_wide.search_hit"は、GROUP BY句の中で出てくるか、集約関数の中で使われなければならない

→ 集約関数（ここではavg）は複数行を1行にまとめる働きがある
→ search_hitだけでは複数行の値を出力してしまう → エラー

空値（NULL）について

データベースの基礎 第3回で学んだ

- 「値が決まっていない」、「値がない」というものもある
 - FAXがない人には、属性「FAX番号」の値はない
 - 社長には、人事情報リレーションの属性「上司」の値はない
 - 平社員には、属性「部下」の値はない
- これらのデータがない状態を表す特殊な属性値が「**空値（NULL）**」である
- あるカラムがNULLであるかどうかは、「カラム名 is null」という条件式によって判定できる

空値（NULL）の例

前に、「access_log_wideのカラムsearch_hitは、request_pathが/searchのとき（つまり検索のとき）だけ値がある」と言ったが、この値がない状態にNULLが使われている

以下のSQL文によって、search_hitがNULLである行を求めることができる

```
SELECT * FROM access_log_wide WHERE search_hit IS NULL;
```

- pgAdmin 4上では、[null]と表記される（小文字表記も許される）

逆に以下のSQL文によって、search_hitがNULLではない行を求めることができる

```
SELECT * FROM access_log_wide WHERE search_hit IS NOT NULL;
```

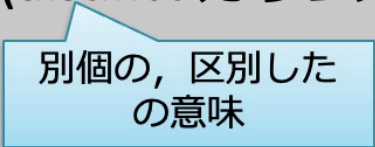
NULLと集約関数

集約関数ではNULLは基本的に無視される

- 例えば、avg(search_hit)を計算する場合、search_hitがNULLの場合は完全に無視され、残りの(NULLでない)行だけの平均が計算される

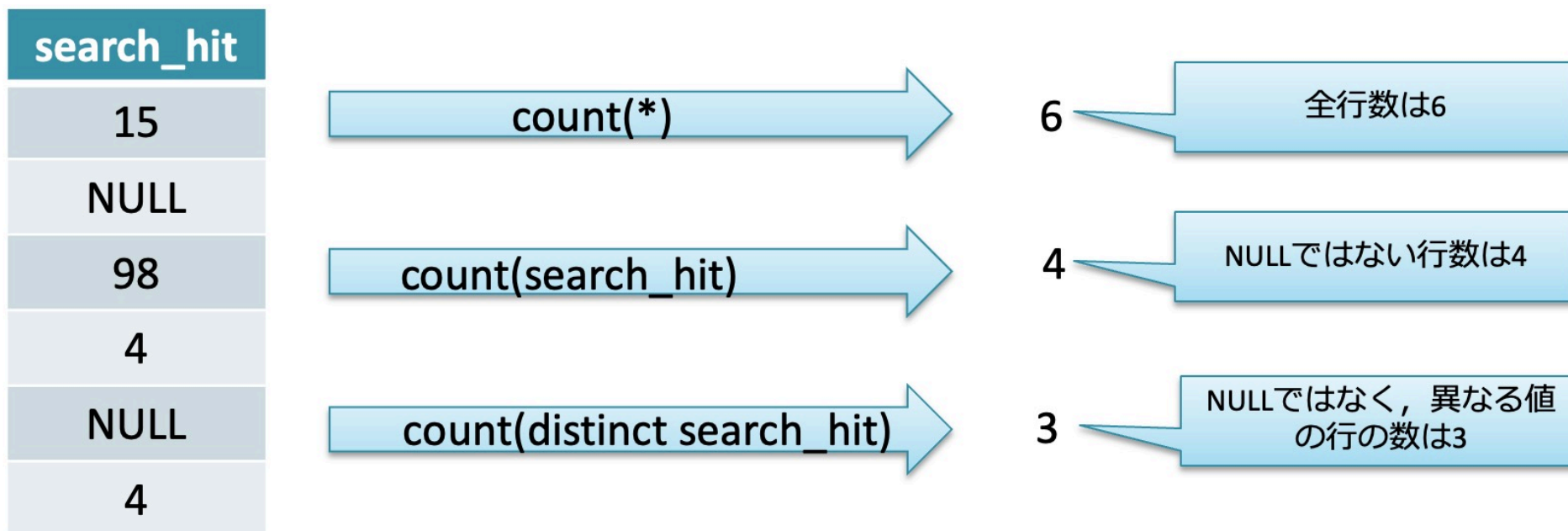
count関数のバリエーション

count関数には次の3つの呼び出し方（使い方）がある

使い方	意味
count(*)	常にテーブルの全行の数を数える
count(カラム名)	そのカラムがNULLではない行の数を数える
count(distinct カラム名) 	そのカラムがNULLではない行について、値の重複を除いた行の数を数える

count関数のバリエーション

例えば，使い方により以下のように結果が変わってくる



ユニークユーザ数の計算

count(distinct カラム名)を使って、ログからユニークユーザ数を計算してみよう

ユニークユーザとは？：特定の期間内にWebサイトにアクセスした「一意の」ユーザ

- 例) 4月のユニークユーザ数は80人, 5月のユニークユーザ数は100人→20人増加
- ユニークユーザ数を求めるとき, ログ上は1人のユーザのログが多く残っているが, その重複を排除して数える必要がある
- 全期間 (期間を限定しない) ユニークユーザ数は, 以下のSQL文で求められる

```
SELECT count(distinct customer_id) FROM access_log_wide;
```

- customer_idは顧客のIDだから, その重複のない数がユニークユーザ数

WHERE句と集約関数の組み合わせ

特定の1日のユニークユーザ数を数えてみよう

→ WHERE句と集約関数を組み合わせれば良い

```
SELECT count(distinct customer_id) FROM access_log_wide
WHERE request_time >= timestamp '2014-12-29 00:00:00'
      AND request_time < timestamp '2014-12-30 00:00:00';
```

- これは2014年12月29日のユニークユーザ数を求めるSQL文

例題：集約関数による単純集計

customersテーブルに対して，以下のようなSQL文を作成せよ

- (1) customer_ageカラムの平均を求めるSQL文
- (2) customer_ageカラムの最大値を求めるSQL文
- (3) customer_ageカラムの値の重複を除いた行の数を求めるSQL文
- (4) customer_genderカラムの値が'F'であるような行について，customer_ageカラムの平均を求めるSQL文

演習：集約関数による単純集計

itemsテーブルに対して、以下のようなSQL文を作成せよ

- (1) item_priceカラムの最小値を求めるSQL文
- (2) shop_idカラムの値の重複を除いた行の数を求めるSQL文
- (3) shop_idカラムの値が6であるような行について、item_priceカラムの平均を求めるSQL文

2. GROUP BY句の利用

どんなときにGROUP BY句を使うか？

集約関数を使うときに、ある単位でグループを作って集約したいときがある

例) ログのテーブルからユーザの単位でアクセス数を数えたい

- これまでの知識だと、WHERE句でcustomer_id（顧客の識別子）を一つ指定して、その行数を数えればよい（count関数）
- customer_id毎にSQL文を作成して、発行する必要がある
 - 例) `SELECT count(*) FROM access_log_wide WHERE customer_id = 18;`
- これを1度のSQL文でまとめて求めたい

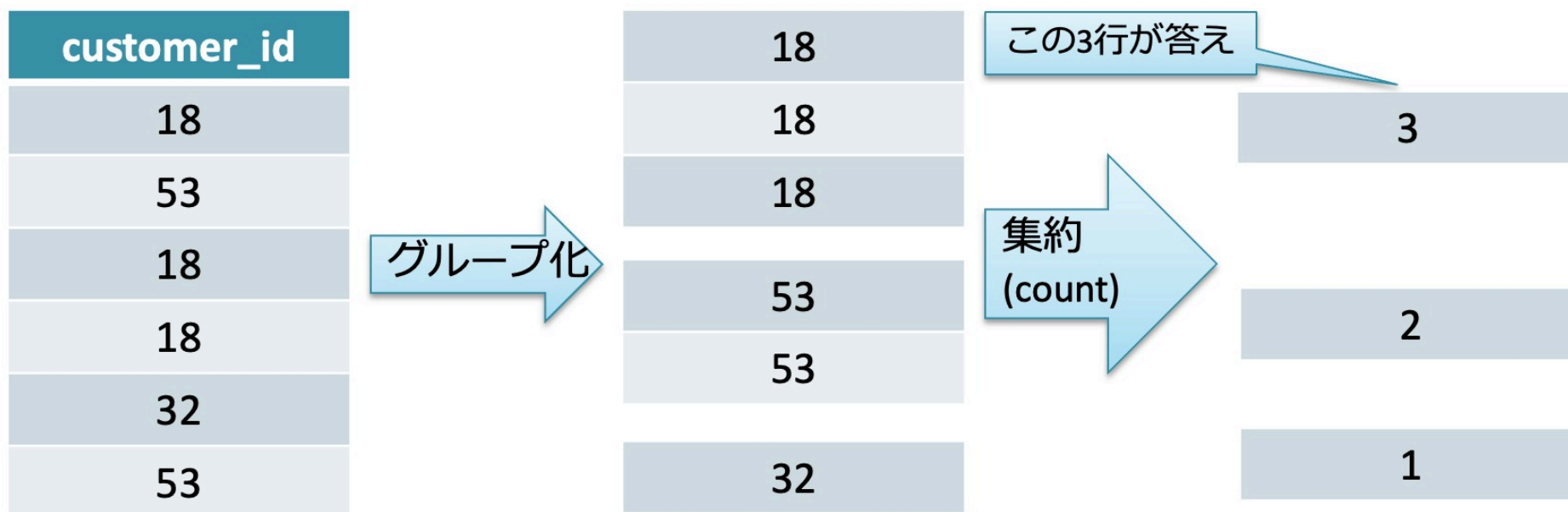
→ **GROUP BY句の利用**

GROUP BY句の働き

例) ユーザ毎のアクセス数を数えるSQL文

```
SELECT count(*) FROM access_log_wide GROUP BY customer_id;
```

- customer_idカラムの値が同じ行でグループを作る（グループ化と呼ぶ）
- グループ毎に集約関数（この場合count）を適用する



GROUP BY句の例

GROUP BY句を使うときには、そのグループを表す値を知るために、GROUP BYで指定したカラムをSELECT句に入れることが多い

- ただし、SELECT句には、GROUP BY句に書いたカラムか、もしくは集約関数のどちらかしか書くことはできない

```
SELECT customer_id, count(*) FROM access_log_wide GROUP BY customer_id;
```

結果

customer_id	count
18	3
53	2
32	1

GROUP BY句の例

GROUP BY句を使えば，月別ユニークユーザを以下のように求めることができる

```
SELECT request_month, count(distinct customer_id) FROM access_log_wide  
GROUP BY request_month;
```

- request_monthカラムには，アクセスがあった月が入っている
- アクセスのあった月の値によってグループ化している
- グループ毎にcustomer_idカラムの重複を排除して，行数を求めている

HAVING句：グループ化した結果をさらに絞り込む

集約結果をある条件によって絞り込みたいときがある

例) アクセス数が1よりも大きいユーザだけ、ユーザ毎のアクセス数を求めたい

WHERE句を使えばよい? → NG

- WHERE句は、GROUP BY句よりも先に実行されてしまう
 - WHERE句で絞り込んだ結果に対して、GROUP BY句が適用される
- 「WHERE count(*) > 1」と指定すると、文法エラーとなる

→ この目的のために使うのが**HAVING句**

- HAVING句は、GROUP BY句によってグループ化されたそれぞれのグループに対して条件を指定できる

HAVING句の使用例

ユーザ毎のアクセス数を数えるSQL文



```
SELECT customer_id, count(*) FROM access_log_wide  
GROUP BY customer_id;
```

customer_idが20よりも大きいユーザについての、ユーザ毎のアクセス数を数えるSQL文



```
SELECT customer_id, count(*) FROM access_log_wide  
GROUP BY customer_id  
HAVING customer_id > 20;
```

アクセス数が1よりも大きいユーザについての、ユーザ毎のアクセス数を数えるSQL文



```
SELECT customer_id, count(*) FROM access_log_wide  
GROUP BY customer_id  
HAVING count(*) > 1;
```

元のテーブル

customer_id
18
53
18
18
32
53



HAVINGなしの結果

customer_id	count
18	3
53	2
32	1

HAVING customer_id > 20の結果

customer_id	count
53	2
32	1

HAVING count(*) > 1の結果

customer_id	count
18	3
53	2

SELECT文の記述順序と実行順序

SELECT文の記述順序は以下である必要がある

1. 取得カラムを指定する（SELECT句）
2. 対象テーブルを指定する（FROM句）
3. 絞り込みの条件を指定する（WHERE句）
4. グループ化の条件を指定する（GROUP BY句）
5. グループ化した後の絞り込み条件を指定する（HAVING句）
6. 並べ替えの条件を指定する（ORDER BY句）
7. 取得する行数を制限する（LIMIT句）

SELECT文の記述順序と実行順序

一方、実際の処理は次の順序で行われる．SELECT句以外は書いてある順番と同じ

1. FROM句 → 検索対象のテーブルが決まる
2. WHERE句 → 行が絞り込まれる
3. GROUP BY句 → グループ化される
4. SELECT句 → カラムが選ばれる
5. HAVING句 → 結果の行が絞り込まれる
6. ORDER BY句 → 並び替えされる
7. LIMIT 句 → 取得される行数が制限される

GROUP BY句でクロス集計？

クロス集計とは，複数の観点（軸）によって，データを表形式に集計する方法である

- Excelではピボットテーブルの機能で求めることができる

例）月毎・ページ毎のユニークユーザ数を表形式で表示したい

→ 下図のような結果イメージ

		ページ毎の観点		
		/	/search	/cart
月毎の観点 (軸)	2015-01	298726	152984	90876
	2015-02	310298	158296	91895
	2015-03	339287	160398	93987
	2015-04	359878	165987	96976

GROUP BY句でクロス集計？

実はSQLは、このようなクロス集計表を直接作ることは得意ではない

- 集計した値を横に並べるのが苦手だからである
- GROUP BY句でカラムを複数指定した集計を行っておき（下図に例を示す），Excelなどにその結果を渡して，クロス集計表を作る方が一般的

```
select
  request_month
  , request_path
  , count(distinct customer_id)
from
  access_log_wide
group by
  request_month
  , request_path
;
```

値は縦に並べる

GROUP BY句に2つのカラム
を指定して分類

request_month	request_path	ユニークユーザー
2015-01	/	298726
2015-01	/search	152984
2015-01	/cart	90876
⋮		
2015-02	/	310298
2015-02	/search	158296
2015-02	/cart	91895
⋮		
2015-03	/	339287
2015-03	/search	160398
2015-03	/cart	93987
⋮		

例題：GROUP BY句の利用

customersテーブルに対して，以下のようなSQL文を作成せよ

(5) customer_genderカラムの値と，customer_genderカラムの値（性別）毎のcustomer_ageカラム（年齢）の平均の値の組をすべてを求めるSQL文

(6) customer_ageカラムの値と，customer_ageカラムの値毎の行数との組（つまり，年齢毎の顧客数）を，customer_ageカラムの値毎の行数が2000より大きい組だけを求めるSQL文

演習：GROUP BY句の利用

itemsテーブルに対して，以下のようなSQL文を作成せよ

(4) item_priceカラムの値が10000より小さい行に対して，shop_idカラムの値と，shop_idカラムの値毎のitem_priceカラムの合計値の組をすべて求めるSQL文

(5) shop_idカラムの値と，shop_idカラムの値毎のitem_priceカラムの合計値の組を，その合計値が100000より小さい組だけを求めるSQL文